ParaMeld：资源感知训练依赖图引导的多智能体混合并行优化

Chenyang Song

上海第二工业大学计算机与信息工程学院，中国

摘要

随着大语言模型的广泛应用，分布式训练的效率优化日益重要。选择高效的混合并行策略需要联合调整并行维度、流水线模块分配和显存优化设置，通常依赖专业经验与反复试运行。现有自动并行方法借助性能模型搜索候选配置，但庞大的配置空间与性能测量开销增加了调优成本。近年来，大语言模型智能体能够通过工具调用和运行反馈自主生成优化策略，为自动配置提供了新的途径。然而，训练吞吐和阶段负载等聚合指标难以揭示跨微批的计算通信依赖，也难以解释关联参数修改引起的执行变化，限制了智能体选择联合策略的依据。为此，本文提出ParaMeld，一种结合训练依赖分析与多智能体协作的混合并行规划方法。ParaMeld 构建资源感知训练依赖图，将计算、通信和存储对象关联到物理 Rank、虚拟分块及微批，通过瓶颈关联子图与反事实执行分析，评价任务成本变化对完整迭代时间的预测影响。多个专职智能体据此选择关联的并行参数、模块范围和运行设置，结合原生约束与条件化联合求解生成完整训练配置。系统进一步利用候选执行差异、成本敏感性分析和真实训练反馈，调整后续策略与测量对象。在双节点 16 张 RTX 5090 上开展的四模型配对实验中，入选联合策略均提高了相对各自随机初始配置的训练吞吐。

关键词：大语言模型；多智能体；混合并行；训练依赖图；资源感知

1 引言

大语言模型的训练需要协同使用多张 GPU，训练效率取决于计算、设备显存和通信资源的共同利用。数据并行、张量并行和流水线并行分别改变样本、张量计算与模型模块的分配；长上下文和混合专家模型还引入上下文并行、专家并行及相应状态管理。不同组合对应不同的本地计算量、通信模式和显存需求[1-2]。为给定模型和集群选择高效策略，需要理解这些机制的关联，并通过试运行检查其实际效果。

自动并行系统利用性能模型和结构化求解减少人工试配。FlexFlow 通过模拟评价并行策略，Piper 和 Alpa 采用分层求解组织多维决策，Galvatron 利用决策树与动态规划搜索 Transformer 的并行组合[3-6]。Aceso 按照性能限制迭代调整配置，Mist 进一步联合优化显存与并行设置[7-8]。这些方法为候选生成提供了有效基础，同时需要处理配置空间、模型适用条件和测量费用之间的取舍。

大语言模型智能体能够读取领域知识、调用系统工具，并根据试验结果修订方案，为训练调优提供了更灵活的决策方式。OptiCo 通过专职智能体协作完成诊断、配置修改和失败处理[9]。针对并行程序优化的 Agent-System Interface 则将任务映射与执行反馈组织为可操作接口[10]。这些工作共同提出了一个进一步的问题：智能体应当获得怎样的训练执行信息，才能把性能现象转化为具有明确作用对象的联合策略？

首先，聚合性能指标压缩了影响修改收益的执行关系。一个阶段的总耗时包含多个微批和虚拟分块的工作，任务还会等待输入、通信以及共享资源。减少某个区间的反向计算，可能同时增加接收区间的计算和激活驻留，并改变其他微批的就绪时刻。仅按最大阶段工作量选择布局，难以判断这些变化对完整迭代的影响。在 Qwen3-14B 的 43

项布局比较中，局部负载与完整执行时间在 854 对非并列比较中的 257 对给出相反排序。这要求搜索能够识别相关任务，并评价其变化沿训练依赖传播后的整体作用。

其次，一项执行目标通常对应多种关联配置。降低 TP 以后，剩余设备可以用于增加 PP 或 DP，两者会产生不同的模块归属、微批数量和存储需求。专家并行还会改变本地专家数量与通信参与者，重计算及状态分片则改变计算、传输和存储之间的取舍[2,11-12]。智能体需要选择共同修改的变量，保留固定训练条件，并根据候选的实际执行变化决定继续调查什么。成本缺项又使部分候选无法直接比较，测量对象的选择因此影响有限预算内能够获得的信息。

ParaMeld 通过训练依赖分析连接性能诊断与策略生成。资源感知训练依赖图按照物理 Rank、虚拟分块和微批组织执行任务，将数据依赖、通信参与关系及存储对象关联到同一配置。分析器从目标任务提取瓶颈关联子图，再通过完整回放评价任务成本扰动的预测影响。智能体据此选择父策略、联合参数和模块范围；原生核验与条件求解将提议转换为完整候选。配置修改后重新构建执行关系，返回模块归属、通信组、等待和显存变化，为下一轮选择提供具体证据。

这一设计同时服务于候选构造和测量选择。需要优化模块边界时，系统可以从完整执行路径提取前后向权重，交给约束工具联合重构布局；需要补充成本时，系统检查共享成本变化是否影响候选排序，帮助评价智能体选择测量对象。四模型配对训练与实际策略轨迹报告完整候选的执行结果，布局排序对照检查局部负载与完整执行目标的差异。九模型及组件实验保留原型系统的汇总口径。

本文的主要贡献包括：

- **面向策略修改的资源感知训练依赖分析**。在统一任务表示中关联模块、微批、物理资源与存储对象，通过瓶颈关联子图和反事实执行分析定位修改对象，并将候选变化映射到具体任务和完整迭代时间。
- **训练依赖引导的多智能体联合规划**。智能体利用执行证据选择关联参数、模块范围和调查预算，结合原生约束与条件求解生成完整策略，并通过执行差异、成本敏感性及训练反馈修订后续搜索。
- **面向 Megatron 训练的系统实现与策略评价**。实现图查询、联合候选生成、性能回放及训练测量，利用四模型配对结果、逐轮策略记录和布局对照分析策略变化，保留原型系统的九模型及组件评测。

2 相关工作

混合并行与流水线调度。GPipe 与 PipeDream 通过微批执行和阶段划分扩展训练规模[13-14]，Megatron-LM 组合张量、流水线和数据并行，并采用交错流水线减少空闲时间[1]。DAPPLE 与 PipeDream-2BW 进一步研究分区、同步及状态保存的协同设计[15-16]。Zero Bubble 通过拆分反向计算组织更紧凑的执行次序[17]，AdaPipe 联合选择重计算与流水线分区[12]。这些研究揭示了模块位置、微批顺序与显存需求之间的联系，构成 ParaMeld 训练依赖分析的执行基础。

自动并行规划。Piper 以两层动态规划组合多维决策，Alpa 分解算子内与算子间并行，Galvatron 利用决策树压缩配置搜索[4-6]。AMP、Metis 和 Sailor 将模型、设备及网络差异纳入自动规划[18-20]。nnScaler 通过约束表达并行变换，NEST 将网络与显存条件纳入设备放置[21-22]。Aceso 依据当前性能限制迭代调整策略，Tessera 利用重叠后的模块组合成本选择流水线分区[7,23]。ParaMeld 利用这些结构化求解思路承接智能体提出的联合修改，并向后续决策返回修改对应的执行证据。

显存管理与专家训练。ZeRO 通过分片减少本地模型状态，ZeRO-Offload 进一步利用 CPU 资源保存和处理状态[11,24]；激活重计算在额外计算与保存需求之间进行选择[12]。Mist 与 ProTrain 根据计算、传输和内存成本联合选择资源管理设置[8,25]。DeepSpeed-MoE 与 MegaBlocks 分别研究专家并行及稀疏专家计算的执行效率[2,26]。ParaMeld 将这些设置保留在完整策略中，检查其与模块布局、通信和微批安排的关联。

训练性能建模。Proteus 通过执行图模拟计算通信重叠和资源竞争，ASTRA-SIM 研究分布式训练的网络及系统模

拟[27-28]。SimAI 联合训练框架、计算与集合通信语义进行性能评价[29]；vTrain 采用剖析驱动模拟，Lumos 利用执行轨迹回放并推算新配置[30-31]。Calculon 提供高层参数化成本模型，Megatron-LM 通信频率研究则将实测通信规律用于异构训练时间预测[32-33]。ParaMeld 在完整回放上增加任务关系查询、成本扰动和候选执行比较，使性能信息能够参与智能体的策略选择。

智能体优化与搜索控制。ReAct 通过推理与工具操作交互求解，Reflexion 利用历史反馈修订后续行为[34-35]。AgentHPO 与 LLAMBO 研究如何利用任务描述和历史试验生成参数候选[36-37]，Optuna 提供自动调优的搜索与试验组织机制[38]。在系统优化中，Agent-System Interface 将操作空间与执行反馈结构化，STELLAR 结合运行日志、参数调查和经验复用优化并行文件系统[10,39]。OptiCo 面向训练配置诊断和建议，LLM-LNS 通过语言模型选择开放变量并调用数值求解器[9,40]。ParaMeld 围绕训练任务的依赖与资源关系组织查询，将联合配置生成、执行变化和后续测量选择连接起来。

3 背景与问题定义

混合并行配置同时决定设备分组、模块归属和微批执行。本节先说明这些选择怎样共同影响训练，再定义 ParaMeld 求解的完整规划问题。

3.1 混合并行与模型结构

张量并行（TP）划分层内张量计算，流水线并行（PP）把连续模型区间分配到不同阶段，数据并行（DP）让多个副本处理不同样本。上下文并行（CP）划分序列，专家并行（EP）分配专家，专家张量并行（ETP）划分专家内部计算。各维度通过进程组确定计算与集合通信的参与者。

流水线还受到微批与调度的影响。全局批量在数据副本和微批之间分解，前向激活需要保存到对应反向任务使用。普通流水线每个物理阶段持有一个模型区间；虚拟流水线（VPP）使同一阶段持有多个区间，并交错执行其微批[1]。模块迁移因而会同时改变设备工作、任务就绪与激活驻留。

模块本身也具有结构差异。稠密 Transformer 的重复层之外，嵌入、输出投影与损失为首尾阶段增加固定工作[1]。MoE 计算依赖被路由的 Token，专家存储则取决于本地持有的全部专家[2,26]。混合模型还组合 Attention 与状态空间计算等不同结构。本研究使用的 Nemotron-H-4B 包含 24 个 Mamba2、24 个 MLP 和 4 个 Attention 模块，分区保持它们的原始顺序。本文考察模型模块的结构差异，实验 GPU 可以同构。

图 1展示完整策略的三个关联部分：全局进程分组、流水线模块布局与运行设置。规划需要共同确定这些选择，使设备分组与具体执行安排相容。

3.2 联合决策的性能影响

Qwen3-8B 的测量提供了具体例子。图 2的 8 卡组固定 CP1、VPP1、全局批 16、微批 1、BF16 和完全重计算，并关闭 P2P 重叠。TP4/PP2/DP1 把 36 层分为两个 18 层阶段，吞吐为 9155.40 token/s，峰值显存为 19.33 GiB。

当 TP 降至 2 时，剩余设备可以用于增加 PP 或 DP。TP2/PP4/DP1 采用四个 9 层阶段，每副本执行 16 个微批；TP2/PP2/DP2 采用两个 18 层阶段，每副本执行 8 个微批。二者吞吐分别为 12029.35 与 12069.75 token/s，显存为 22.33 与 27.06 GiB。同一 TP 修改形成相近吞吐，却具有不同的存储代价和后续调优空间。12 卡组还包含 CP、DP 与 VPP 的联动。生成器需要展开这些完整实现，才能判断一项修改的效果。

3.3 完整策略的规划目标

规划输入为模型 $\mathcal{M}$、设备拓扑 $\mathcal{D}$、固定训练负载 $\mathcal{W}$ 和搜索预算 $\mathcal{U}$。训练负载包括检查点、数据、全局批量、序列长度、精度及优化器语义。预算限制智能体调用、性能预测、GPU 测量和搜索轮次。

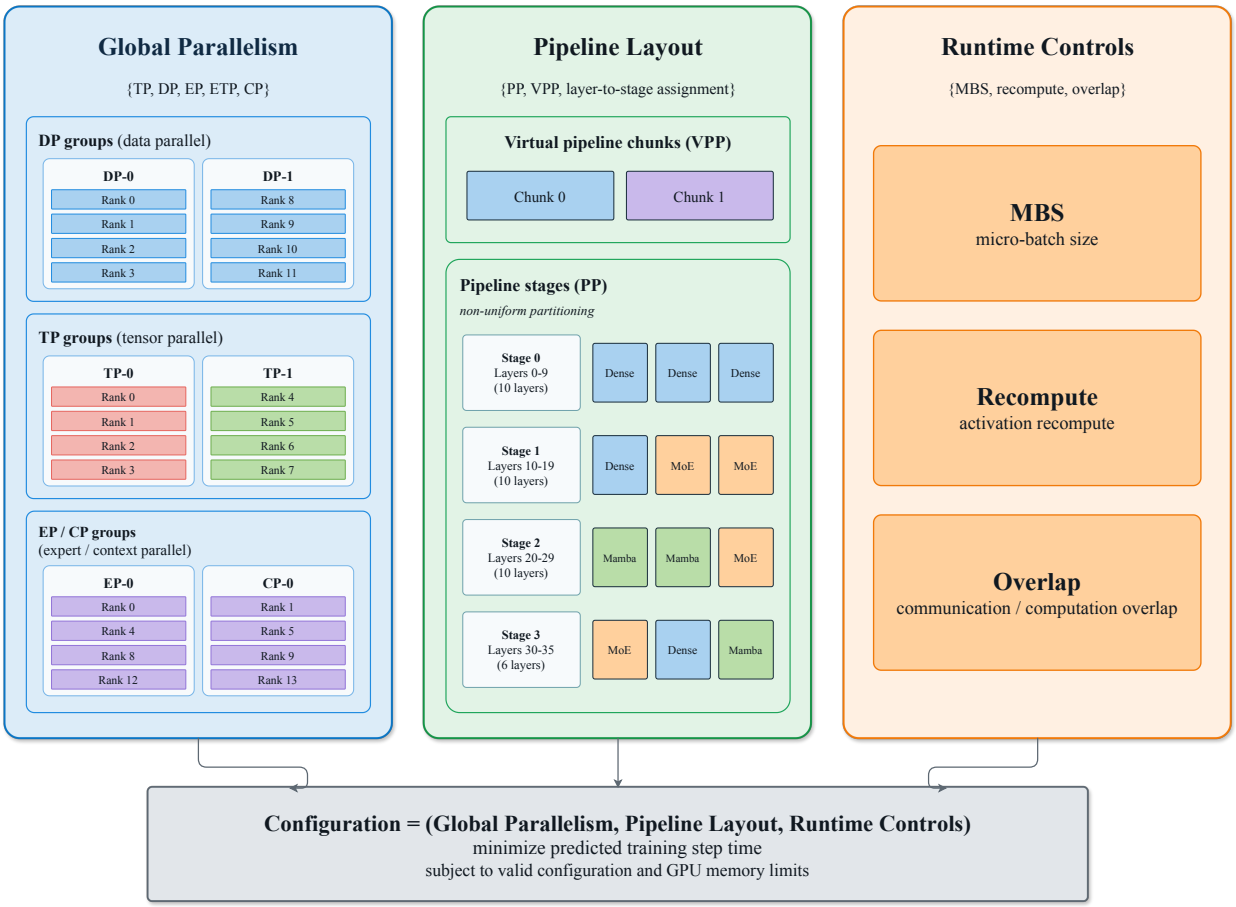


图 **1**　混合并行策略的决策空间。全局并行包括 TP、DP、EP、ETP 与 CP，流水线布局包括 PP、VPP 与非均匀模块分区，运行设置包括微批、重计算和通信重叠。图中的设备分组与模块数量为示意。

完整策略表示为

$$\Theta = (H, \Lambda, Z), \tag{1}$$

其中，H 包括并行维度与进程放置，Λ 包括模块区间、微批、调度、重计算及通信重叠，Z 指定执行后端和训练状态分片。仅当这些设置共同确定时，候选才对应具体的执行配置。

设 GPU 数量为 N、可分区模块数量为 D。TP、PP、稠密 DP 和 CP 的并行度分别记为 t, q, d, c，EP、ETP 和专家 DP 记为 e, t_e, d_e。每张 GPU 运行一个 Rank，进程组满足

$$tcdq = N, \qquad t_e e d_e q = N. \tag{2}$$

全局批量 B、微批大小 b、每个稠密副本的微批数量 n_μ 及 VPP 大小 v 满足

$$B = dbn_\mu, \qquad \sum_{j=0}^{qv-1} x_j = D, \quad x_j \geq 1. \tag{3}$$

$\mathbf{x}$ 按模型顺序给出各区间的模块数量，所有区间连续覆盖模型。专家副本按专家进程组处理；稠密模型使用 $e = 1$ 及 $t_e = t$。模型与后端还要求张量整除、调度支持及设备映射相容。

记满足这些约束的策略集合为 $\mathcal{F}$，实测迭代时间为 $T(\Theta)$，Rank r 的显存需求与容量分别为 $M_r(\Theta)$ 和 C_r。规划目标为

$$\min_{\Theta \in \mathcal{F}} T(\Theta), \qquad M_r(\Theta) \leq C_r \quad \forall r. \tag{4}$$

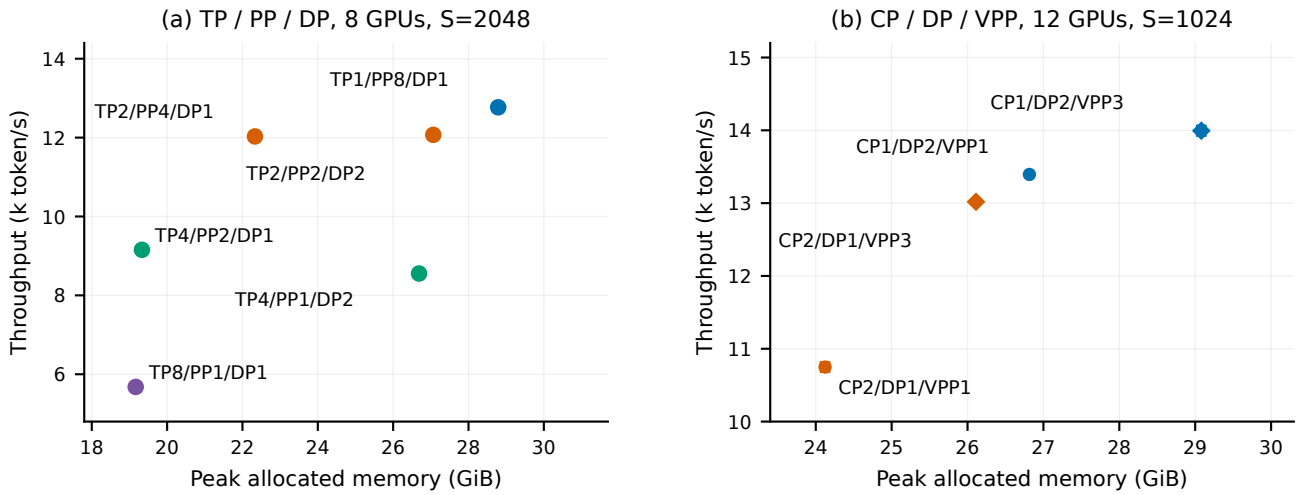


图 2　Qwen3-8B 不同混合并行配置的吞吐与显存。两组负载分别比较；右图误差线为 4 次运行的样本标准差。完整配置及测量窗口见附录表 E.1。

ParaMeld 在预算内生成和评价候选，分别维护预测结果与成功训练策略。序列长度记为 S，预测显存采用第 4.1节的对象存活模型。

4　训练依赖分析驱动的多智能体混合并行规划

混合并行策略同时决定工作分配与执行依赖。阶段总工作量描述设备承担多少计算，训练依赖则决定这些工作何时能够执行，以及修改会影响哪些后续任务。ParaMeld 将两者保存在同一执行表示中，使智能体能够围绕具体任务调查原因、选择关联变量，并根据完整候选的结果继续调整。

方法包含训练依赖分析与多智能体联合规划。前者提供资源感知回放、瓶颈关联子图及反事实成本分析；后者使用这些证据生成完整策略或条件修改，并组织合法性检查、候选评价和测量。系统同时保存本轮试验与已有最好策略，实际执行结果进入下一轮上下文。

图 3展示四个智能体与分析、求解及训练工具之间的关系。算法 1给出一次调查从执行证据到联合修改，再到实际验证和反馈修订的过程。

4.1　资源感知训练依赖图

给定完整策略 Θ，资源感知训练依赖图记为 $G_\Theta = (V_\Theta, E_\Theta, \mathcal{R}_\Theta, \rho_\Theta, \mathcal{B}_\Theta)$。$V_\Theta$ 表示任务，E_Θ 表示数据、调度及同步依赖，$\mathcal{R}_\Theta$ 为计算和通信资源，ρ_Θ 记录任务的资源需求，$\mathcal{B}_\Theta$ 保存相关存储对象及其使用依赖。每项计算任务对应指定 Rank、虚拟分块和微批上的前向或反向工作，成本由相容模块测量聚合。流水线传输、梯度同步与优化器更新采用显式任务表示；FSDP 配置另外包含参数收集、重新分片及 ReduceScatter。

图中的依赖包括阶段之间的激活与梯度传递、原生调度规定的微批次序及参数同步条件。区间 j 映射到物理阶段 $j \bmod q$ 和虚拟编号 $\lfloor j/q \rfloor$[1]。多个区间共享设备时，其计算资源共同计入回放，各自的前后向依赖分别保留。模型结构决定区间内部成本，Falcon-H1 等混合结构保留原生分支及汇合关系。

通信同时依赖进程组与物理放置。并行维度决定参与成员和消息大小，设备映射决定节点内或节点间路径。系统按式 (2)分别建立稠密和专家分组，并在配置变化后重新确定相应传输。模块复合测量已经包含的 TP、CP 或 EP 通信留在节点成本内，显式流水线传输和适用的梯度同步分别计入。

显存评价沿用同一执行关系。前向激活保存至对应反向任务使用，传输缓冲保存至通信及后续使用结束，临时

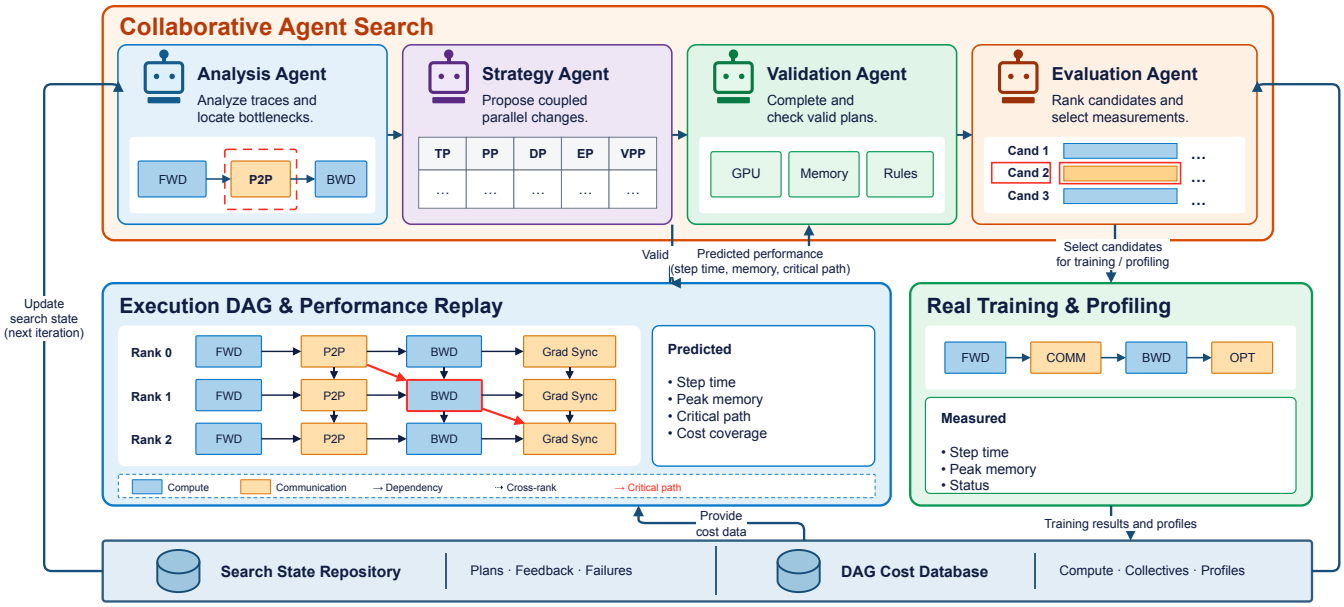


图 3　ParaMeld 的四智能体框架及执行评价与训练反馈。

收集参数保存至使用和重新分片完成。边界迁移改变这些对象的持有设备及存活时段，因此完整候选重新建立计算、传输和释放之间的联系。MoE 计算采用相应路由条件，持久参数存储覆盖本地持有的全部专家。

图的成本来自与实现、设备、形状、并行维度、微批、序列、精度及重计算相容的记录。直接测量优先，计算估计使用 Roofline 模型；通信估计结合消息大小、集合算法和拓扑条件[28]。整段 CUDA 计时按实际采集范围匹配，计算与通信按照各自覆盖范围组合；实测、估计及缺测状态分别保存。

回放依据任务就绪和资源可用性推进。单个 Rank 上的计算共享设备资源，关闭 P2P 重叠时传输也占用该资源；共享路径的并发通信共同使用带宽，计算通信并发采用适用测量提供的减速系数。完成事件释放资源并更新后继任务，最终得到迭代时间。根据回放中的依赖和资源等待[27]，系统恢复决定各事件完成时刻的前驱关系，为路径提取提供已经确定的执行次序。

Rank r 的预测显存为

$$\widehat{M}_r = P_r + G_r + O_r + \max_{\tau} \sum_{a \in \mathcal{A}_r(\tau)} |a| + R_r. \tag{5}$$

P_r、G_r 和 O_r 分别为本地参数、梯度与优化器状态，按照模块归属和分片规则计算。$\mathcal{A}_r(\tau)$ 包含存活激活、流水线缓冲和临时收集参数的新增存储，R_r 为工作区及实测附加占用。对象按存储身份去重，持久分片和临时收集分别处理；MoE 缓冲依据适用的路由及分发数量计算。工作区或缓冲信息不足的候选保留容量未知状态。已经确认的必需存储下界超出容量时，搜索器排除该候选；成功训练策略另外登记。

组件校准采用相容实测与估计的比值。迭代级校准在参照实测时间上增加候选与参照的模拟时间差，适用于两者残余误差相近的情形。校准条件包括模型、硬件、负载、并行维度、微批分组、精度、重计算、调度、映射和状态分片；条件变化后重新匹配测量。

4.2　瓶颈关联子图与反事实执行分析

阶段或任务耗时只能确定调查起点。为了解释其等待来源，分析器以目标任务集合 U 为起点，在指定深度内查询前驱与后继，同时纳入同一通信实例的参与任务和共享资源的相关任务。具有预测起止时刻时，共享资源关系进一步限定为执行时段重叠；时间信息缺失时返回潜在相关任务。查询结果保留任务进入子图的原因、模块归属、Rank 及微批身份，并记录子图外部的直接输入依赖。

这种提取方式保留了调查目标与其他设备工作的联系。例如，一个接收任务的等待可能来自发送阶段尚未完成计算，也可能来自共享传输资源；两个虚拟分块虽然持有不同模块，仍可能在同一 GPU 上竞争计算。智能体通过子图继续查询这些关系，无需将全部微批任务同时放入上下文。查询采用任务索引、分页和字符配额，当前策略及重要历史结果保持在紧凑摘要中。

为判断目标工作对迭代时间的影响，系统在保持完整依赖与资源需求的条件下缩短所选任务成本，再执行完整回放。对任务集合 U 和比例 $0 < \epsilon \leq 1$，定义

$$\Delta_\epsilon(U;\Theta) = \widehat{T}(\Theta) - \widehat{T}(\Theta \mid d_v \leftarrow (1-\epsilon)d_v,\ v \in U)\,. \tag{6}$$

其中 d_v 为任务的基准成本，返回结果同时包含原始与扰动后的关键任务。若局部成本下降被其他依赖或资源等待吸收，完整时间的变化会小于局部节省。智能体据此比较调查对象，再查询能够影响这些任务的合法策略操作。

式 (6)描述固定图上的成本扰动。具体并行修改还可能改变任务数量、模块归属、通信组和存储关系，系统根据新配置重新构建执行图并评价这些共同变化。父子策略比较由此连接目标任务、实际配置修改及其执行结果。

4.3 执行证据引导的联合策略生成

四个智能体围绕同一策略档案协作。分析智能体读取模块结构、完整配置、任务摘要及子图查询，形成带有任务依据的优化假设；策略智能体选择父策略、关联参数、模块范围和调查配额。核验智能体组织原生约束检查与候选展开，评价智能体安排回放、组件测量和训练验证。专职角色共享候选及结果，使诊断依据能够进入实际配置修改。

联合变量的选择来自目标任务与模型结构的对应关系。减少 TP 通信需要检查增加的单卡计算和存储，并重新选择其余设备的分配；迁移输出阶段的模块需要检查接收区间及其共驻分块；修改 EP 需要同时考虑本地专家数量、ETP 和通信参与者。修改预览返回受影响的计算形状、模块、通信组和缓冲区依赖，智能体据此确定共同修改的设置及保留条件。

策略智能体可以直接提交完整配置，也可以通过修改意图 $I = (op, V, A, P, R)$ 提出条件调查。op 指定修改类型，V 指定取值，A 给出允许共同改变的变量，P 保留父策略条件，R 限定允许范围。每项提议还包含目标任务、证据来源和预期执行变化。完整配置经过核验后可以直接评价和训练；条件提议由工具展开多个相容实现。

例如，第 5.7节的 Qwen3-14B 轨迹中，智能体在查询局部反向成本后共同选择模块接收位置与选择性重计算。布局改变工作归属，重计算设置改变相应工作量和激活保存，两项选择形成同一个完整候选。后续训练将该联合修改的实际吞吐与父策略比较，并把结果提供给下一次调用。

4.4 原生约束与条件化联合求解

核验工具检查设备分解、张量整除、批量分解、模块覆盖和调度支持。对条件提议，CP-SAT 保持 Agent 指定的目标与保留条件，选择附加变化较少的合法补全，再排除已有组合继续生成候选。联合候选的全局参数、模块布局与状态分片均按照实际生效设置登记。

条件分区工具在给定 TP、PP、CP、DP、EP、ETP 及设备映射后，按照允许范围组合 VPP、微批、分组、重计算和状态分片。每项组合使用按阶段扩展的动态规划结构构造连续模块区间，状态记录已覆盖位置、区间归属及成本可用时各物理阶段的前向、反向和参数状态成本。到达同一端点的状态按评分保留有限数量，并受到扩展次数与时间配额限制。局部评分使用相容模块成本，缺少成本时使用区间数量；完整候选随后接受执行评价。提供相容完整续接及回放配额时，完整评价也参与部分状态的扩展与保留。

未开放的设置继承父策略，PP 或 VPP 变化后重新解析 P2P 重叠、微批和重计算等运行条件。相同并行度下的不同生效设置分别匹配成本。全部候选保持检查点、数据、全局批量、序列长度、精度和优化器语义，冲突返回具体

条件。缺少适用成本的完整策略保留测量需求，并可按剩余训练预算接受实际验证。

4.5 执行路径引导的模块边界求解

执行路径引擎在具备适用逐模块成本和完整执行评价时，对给定并行与运行条件重构边界。它将 D 个有序模块划分为 $K = PP \cdot VPP$ 个连续区间，边界满足 $0 = b_0 < b_1 < \cdots < b_K = D$。物理阶段总工作量合并了同一设备持有的多个区间，区间身份则保留它们在微批调度中的位置。该引擎利用这种差异估计模块迁移对执行路径的影响，独立预测搜索实例见附录 C。

考虑一条包含多个微批的执行路径。若路径经过某区间的前向任务多次，该区间减少的单次前向工作会在路径上重复累积；反向工作具有相应的累计次数。模块移入另一区间以后，新增工作则按照接收区间的次数累计。同一物理阶段上的区间可以具有不同次数，因此模块迁移的作用由其成本、迁出位置和接收位置共同决定。多条路径同时参与目标，使求解器在缩短当前较长路径时也考虑其他路径的增长。

搜索首先选择一项具有相同执行条件的完整布局作为参照。参照来自相容父布局、智能体提交的完整种子或已有策略；PP/VPP 变化时重新建立相应调度。系统在回放依赖上加入排他资源的实际执行顺序，恢复事件之间的资源等待关系。随后正向计算到达每个事件的最长前缀，反向计算从该事件出发的最长后缀，将二者连接为经过该事件的参照路径。路径中每项前后向任务均增加对应区间的累计次数。系数相同的路径保留较大的常数项，提取代价为 $O(K|V| + |E|)$。

设 $F_j(\mathbf{b})$ 和 $B_j(\mathbf{b})$ 为边界 $\mathbf{b}$ 下区间 j 的逐模块前向、反向成本之和，a^F_{pj} 与 a^B_{pj} 为路径 p 中对应任务的出现次数。生成目标为

$$\widetilde{T}(\mathbf{b}) = \max_{p \in \mathcal{P}} \left[c_p + \sum_{j=0}^{K-1} \left(a^F_{pj} F_j(\mathbf{b}) + a^B_{pj} B_j(\mathbf{b}) \right) \right]. \tag{7}$$

c_p 等于参照路径时间减去该路径的模块成本项，保留通信、固定任务及其余执行代价。在参照布局上，式 (7)复现回放的完整时间；边界变化时，它估计参照路径结构下的工作迁移。外层最大值同时约束多条路径，迁出收益和接收代价共同进入每条路径的计算。两个区间即使共享物理阶段，也保留各自的前后向系数。

为求解这一目标，生成器预先累计模型顺序上的前后向模块成本，使用相邻边界的前缀和之差表示 F_j 与 B_j。CP-SAT 共同选择开放范围内的全部边界，约束每个区间非空、模型连续覆盖及冻结模块保持归属。式 (7)作为主目标，总边界位移用于打破并列。一次求解由此可以同时迁移多个边界，比较不同区间之间相互补偿的完整布局。已评价布局加入排除集合，后续求解继续生成不同候选。

模块迁移也可能改变通信就绪、资源次序和显存峰值。这些变化通过完整回放重新计算。只有取得有效时间、容量下界未超限且完整时间改善的候选，才替换当前搜索中心；容量未知状态随结果保留。新中心重新产生路径及其权重，下一次求解随之改变。例如，原迁出区间缩短以后，接收区间可能进入新的最长路径，其前后向累计次数便参与下一次边界选择。生成目标与执行结果通过这一更新持续联系。

布局搜索在固定执行条件内使用路径代理，完整回放负责候选选择。智能体改变 TP、PP、VPP、微批或重计算以后，生成器重新匹配成本并提取参照路径，保持目标与新调度相容。该算法采用变量冻结与联合重构的求解方式[40]，其边界目标及更新依据来自具体训练执行。条件分区工具还支持用合法完整布局接续部分状态进行评价，候选来源随调用记录保存。

4.6 执行反馈、测量选择与迭代修订

一次联合修改的总时间由多个变化共同形成。降低 TP 可以减少集合通信，同时增加本地模块工作；模块迁移可以缩短原区间，同时增加接收区间的工作和激活驻留。下一轮需要知道各项变化对应的执行对象。ParaMeld 按稳定模块与操作标识比较父子策略，将配置修改关联到计算形状、模块归属、通信成员和保存对象，返回相关任务、逐

Rank 计算占用、显式通信暴露及容量状态。

这些证据在两个决策层次发挥作用。固定条件内，所选分区引擎继续生成候选；启用路径引擎时，生成器按第 4.5节更新目标。新的智能体调用读取已完成候选及执行摘要，决定继承哪些设置、开放哪些关联变量，以及继续调查哪个父策略。一次调用也可以提交多项调查，随后由控制器按次序执行。系统分别记录新的模型提议与同次提议的工具续接，附录 C给出后一种情况的完整实例。

策略档案中的当前最好实测结果独立于本轮试验候选。新配置完成规定测量、通过有效性检查且吞吐超过已有最好值时，更新实测记录；性能下降或执行失败时保留已有结果。Agent 仍可选择其他历史父策略继续调查，以探索当前最好配置附近尚未覆盖的参数组合。预测记录按其成本身份与容量状态单独排序，尚无成功训练记录时保留缺失状态。

评价智能体还需要选择补测对象。工具按操作形状、通信组和消息大小汇集缺项，记录每项成本影响的候选。对具备基准预测的配置，系统分别采用共享成本的较低和较高取值回放全部相关候选，比较最好策略、前 k 项的成员与次序是否变化。同一成本在不同候选中共同改变，保持其共享测量来源。

测量建议优先考虑可能改变最好策略的成本，其次考虑前 k 项的次序及成员变化，再比较预计测量费用。评价智能体读取这些排序变化、成本复用范围及剩余预算，选择组件测量、完整训练或继续调查其他策略。新增测量后，相关候选使用更新成本重新评价。高低取值属于敏感性情景，统计置信区间由实际重复测量另行提供。

失败按其完整执行条件进入反馈。原生冲突、容量预测、实际 OOM 和执行错误分别关联布局、微批、调度与重计算信息。后续调用通过摘要和任务索引读取相关结果，核验工具再次解析所选父策略的生效设置。Lightning 的容量修复实例包含多轮分片、卸载与并行组合提议，执行结果用于继续修订搜索条件，具体记录见附录 B.2。

4.7 实现与预算控制

ParaMeld 基于 Megatron-LM 实现策略生成与训练执行。模型分区保留原始模块顺序，候选按照模块身份加载检查点并构造对应进程组。训练采用分布式优化器；普通流水线使用非交错 1F1B，虚拟流水线使用交错 1F1B。每项候选独立初始化优化器，测量窗口覆盖前向、反向、同步和参数更新，同时检查损失、梯度范数及更新状态。

GPU 训练轨迹按实际调用记录完整策略核验、条件展开及训练测量。执行路径引擎的独立布局构造实例采用附录 C中的 CPU 预测搜索，预测值与实测吞吐分别归属各自的运行协议。

每批 Agent 输出可以包含多项有序调查，记录中的调查批次通常为 2 至 4 项，每项至多提供两项完整布局种子。条件分区工具按端点维护候选宽度及扩展配额。路径引擎采用整数微秒、单工作线程和至多 1.5 秒的单次求解配额，首次求解覆盖全部授权边界，后续第 k 次构造的总位移上限为 $\min(DK, 2^k)$，$k \geq 1$。这些设置随实际分区引擎分别登记。

控制器根据已完成调查的耗时与新增预测数量分配剩余配额。相同必要成本连续缺失两次时停止该项新评价；剩余时间能够容纳已有智能体调用耗时中位数及至少两次预测时，才启动下一项调用。全部模型调用、工具执行与 GPU 使用均计入费用。最终选择采用截止前完成的结果，并保留智能体提议、约束补全与布局搜索各自产生的候选。

5 实验评价

我们评价 ParaMeld 生成的完整策略及其执行结果。四模型配对测量比较训练吞吐与显存，布局排序对照检查生成目标，真实策略轨迹解释智能体提出的修改如何经过工具执行。原型系统的九模型汇总与组件比较保留其原始统计口径。

算法 1 训练依赖分析驱动的多智能体联合策略搜索

输入： 模型结构、设备拓扑、固定负载、原生约束、成本与预算

输出： 最好实测策略、最好预测候选及其执行状态

核验初始策略并建立策略档案与执行记录
while 搜索预算允许 **do**
    分析智能体查询目标任务的依赖、通信参与者及共享资源，提取关联子图
    有相容成本与回放配额时，评价目标任务成本扰动对完整时间的影响
    策略智能体选择父策略，查询修改影响，提交联合候选与调查配额
    核验完整取值或补全条件意图，解析原生生效设置
    for 每项提议 **do**
        if Agent 已经指定完整策略 **then**
            核验并保留该完整候选
        else
            补全全局结构，在固定条件下展开相容布局与状态分片
            调用条件分区工具，在配额内生成完整候选
        end if
        if 需要继续构造布局，且成本与配额允许 **then**
            按所选引擎继续展开；路径引擎使用式 (7)并回放更新中心
        end if
        评价已有完整候选，登记预测、容量状态及成本缺项
        比较父子策略的执行变化，并检查共享成本变化对候选排序的影响
        评价智能体依据排序敏感性、覆盖范围和费用选择实际训练或组件测量
        有效训练取得更高吞吐时更新最好实测记录，其余结果进入反馈
    end for
    更新策略档案与剩余预算，将执行差异返回智能体
end while
return 已有最好实测记录及截止前最好预测候选

5.1 实验设置

实验平台包含两个节点、16 张 NVIDIA RTX 5090 GPU，每张显存 32 GB。节点间通过 400 Gb/s InfiniBand 连接，节点内部使用 PCIe 5.0。四模型测量记录的 PyTorch 版本为 2.12.0a0+5aff3928d8.nv26.05，CUDA 为 13.2，执行后端为 Megatron DDP。九模型原型汇总采用 BF16，随机初始配置实验采用 FP16 计算与 FP32 优化器状态。各项配对比较固定检查点、数据、精度、Adam 设置与全局批 16。

主表分别比较默认配置、人工配置与 ParaMeld。默认配置使用 Megatron 入口的参考并行组合与近似均匀分区；人工和系统从相同配置出发，均允许最多 12 次训练尝试，并可调整并行维度、VPP、微批、重计算、重叠与模块边界。历史追加搜索采用共同档案中的最好可评价策略，独立训练案例采用各自登记的共同基线。

九模型训练原型的协议记录预热 20 次、测量 50 次，每项最终配置运行 3 次，结果采用系统汇总。吞吐为 $Q = BS/T$，模型 FLOPs 利用率（MFU）按每 GPU 209.5 TFLOP/s 归一化。MoE 和混合模型按激活参数估算，MFU 用于同一模型内比较。训练原型的智能体设置记录为温度 0，每轮最多 6 次工具调用；消融使用 3 个搜索种子。各项数据来源和统计范围见第 5.8节。

表 1　稠密模型的原型评测归档汇总。

模型	策略来源	MFU (%)	吞吐 (token/s)	显存 (GiB)
MiMo-7B	默认配置	24.5	17133	13.6
	人工配置	24.6	17231	14.6
	ParaMeld	**27.1**	**18932**	11.0
Qwen3-8B	默认配置	46.1	31439	15.8
	人工配置	46.6	31751	14.1
	ParaMeld	**51.7**	**35212**	23.7
Mistral-NeMo-12B	默认配置	32.3	15162	25.5
	人工配置	36.1	16912	24.8
	ParaMeld	**36.7**	**17209**	22.9
Qwen3-14B	默认配置	42.1	15886	21.5
	人工配置	47.5	17913	25.2
	ParaMeld	**48.6**	**18348**	25.2

双节点 16 卡；汇总协议为每项 3 次运行。数据来源和记录范围见第 5.8节。

表 2　MoE 与混合结构模型的原型评测归档汇总。

模型	策略来源	MFU (%)	吞吐 (token/s)	显存 (GiB)
Qwen3-30B-A3B	默认配置	2.0	3377	28.2
	人工配置	2.3	3895	27.8
	ParaMeld	**2.4**	**4088**	28.3
Moonlight-16B-A3B	默认配置	8.4	17644	22.5
	人工配置	9.5	20072	24.1
	ParaMeld	**9.8**	**20643**	26.8
Nemotron-H-4B	默认配置	—	53841	21.1
	人工配置	—	55320	18.9
	ParaMeld	—	**58687**	19.3
Falcon-H1-7B	默认配置	30.7	24554	20.6
	人工配置	32.2	25736	19.1
	ParaMeld	**34.1**	**27255**	19.1
Lightning-30B-A3B	默认配置	4.3	5998	28.6
	人工配置	5.2	7156	22.9
	ParaMeld	**5.3**	**7297**	23.0

归档汇总报告双节点 16 卡、每项 3 次运行；按激活参数估算的 MFU 用于同一模型内比较。横线表示缺少一致的 FLOPs 计算记录。

5.2　训练性能

表 1与表 2分别报告 4 种稠密模型和 5 种 MoE 或混合模型。吞吐提升为 $100(Q_{\text{ParaMeld}}/Q_{\text{base}} - 1)\%$，参照对应默认或人工配置。

归档表按原有汇总计算，相对默认与人工配置的算术平均提升分别为 14.6% 和 5.2%。这些数值描述原型完整系统的比较。后续配对测量保留各项生效配置与逐 Rank 记录，用于检查具体策略修改。Lightning 的同条件 PVC 复核结果见附录 B.1。

表 3　两个 Qwen 模型上的原型系统比较归档。

模型	方法	吞吐 (token/s)	显存 (GiB)	搜索 GPU 小时
Qwen3-14B	Galvatron	8901	22.6	1.42
	Megatron 1F1B	12113	27.4	–
	人工配置	17913	25.2	6.58
	ParaMeld	**18348**	25.2	2.74
Qwen3-30B-A3B	Galvatron	2249	27.1	1.96
	Megatron 1F1B	3606	28.2	–
	人工配置	3895	27.8	9.84
	ParaMeld	**4088**	28.3	4.12

记录为双节点 16 卡、3 次独立运行平均值。搜索费用包含训练、测量和失败运行；横线表示固定配置无搜索。

表 3在两个 Qwen 模型上比较 Galvatron[6]、固定 Megatron 交错配置、人工调优与 ParaMeld。各组固定训练负载和精度，每 GPU 容量限制为 31 GiB。Galvatron 搜索 TP、PP、DP、微批与重计算，MoE 组的 EP 固定为 4。固定 Megatron 配置在 Qwen3-14B 上为 TP4/PP4/VPP2，在 Qwen3-30B-A3B 上为 TP1/PP4/CP2/EP4/VPP2；ParaMeld 同时调查相关并行维度及模块边界。

5.3　随机初始配置下的联合策略优化

表 4报告独立的 FP16 计算、FP32 优化器状态实验。四个模型分别从本次随机合法配置出发，智能体可以读取本次搜索产生的配置和反馈。实验固定 16 张 GPU、序列 2048 和全局批 16，每项配置预热 5 次、测量 30 次成功更新。吞吐按最慢 Rank 的平均迭代时间计算，显存采用全部 Rank 的最大已分配值；每项配置对应一次完整运行。

图 4比较四个模型调优前后的吞吐和显存。ParaMeld 生成的联合策略提高吞吐 11.72%–61.25%，对应不同的计算与存储取舍。Qwen3-8B 的变化最明显：在保持两个流水线阶段的条件下，策略将 TP 从 8 降至 4、DP 从 1 增至 2，并采用选择性重计算。其模块布局保持不变，性能差异来自完整并行与运行配置的共同调整。

Qwen3-14B 将流水线阶段由 8 个减至 4 个，同时增加 TP 并调整重计算。该策略在每阶段承担更多模块的条件下，将峰值已分配显存降低 9.70 GiB，吞吐提高 11.72%。MiMo 联合调整虚拟流水线和重计算范围，Moonlight 联合调整微批与模块归属；两者分别提高 18.46% 和 13.54%。这些结果对应全局并行、调度和存储设置的共同选择，关键配置见附录表 A.1。

表 4　随机初始配置下的四模型联合策略测量。计算与模型参数采用 FP16，主参数、主梯度和 Adam 状态采用 FP32。

模型	策略	MFU(%)	吞吐 (token/s)	显存 (GiB)	吞吐提升 (%)
MiMo-7B	随机初始	19.91	13656.05	18.24	—
	ParaMeld	23.58	**16177.26**	22.82	18.46
Qwen3-8B	随机初始	14.33	10172.26	9.84	—
	ParaMeld	23.11	**16403.23**	11.94	61.25
Qwen3-14B	随机初始	25.93	10053.36	29.35	—
	ParaMeld	28.97	**11231.79**	19.65	11.72
Moonlight-16B-A3B	随机初始	7.04	14473.16	26.52	—
	ParaMeld	7.99	**16433.23**	26.76	13.54

提升相对各模型本次随机初始配置，MFU 按原始记录的模型 FLOPs 及每 GPU 209.5 TFLOP/s 计算。全部配置完成 35 次成功更新，测量窗口为其中后 30 次。

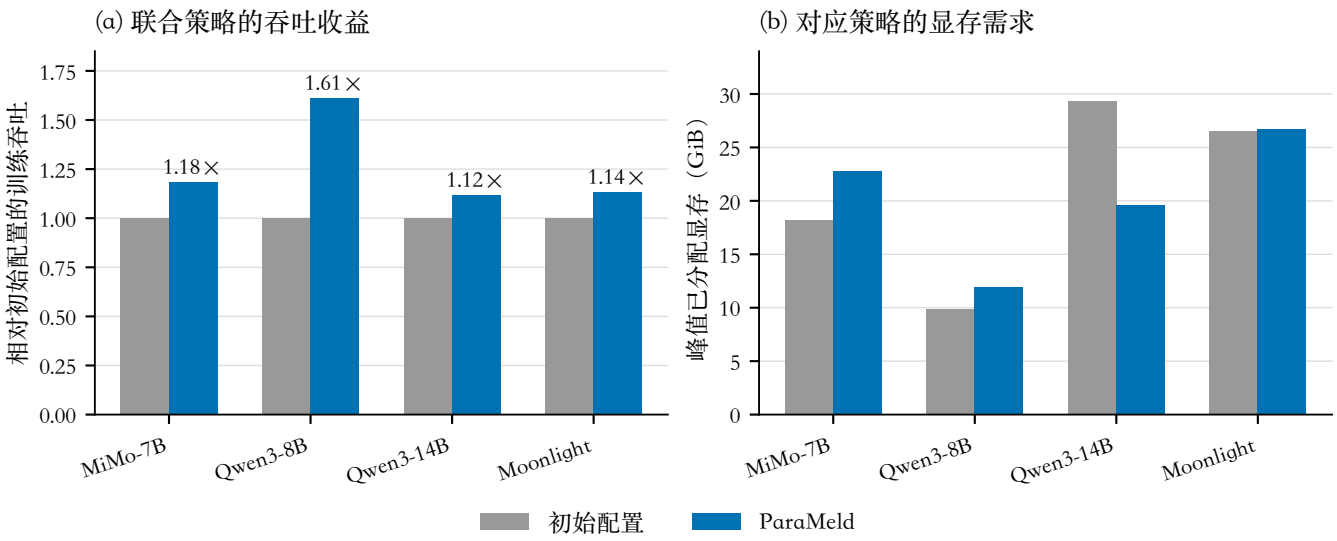


图 4　四模型联合策略的实测吞吐与显存。吞吐按各模型的随机初始配置归一化，各配置独立测量一次。

5.4　智能体组件的作用

我们使用 MiMo-7B、Nemotron-H-4B 和 Qwen3-14B 考察执行证据、独立分析和角色分工的作用。每模型每设置使用 3 个种子，每设置共 9 次搜索，最多 12 次训练尝试。目标固定为各模型默认吞吐的 1.08 倍。组件设置共享原有分区器与预测器，分别移除独立分析角色、将全部职责交给单个智能体，或移除提供给智能体的模块成本与执行依赖。

图 5给出组件比较的结果质量与条件费用。移除执行证据后，平均提升由 11.7% 降至 2.5%，9 次均未达标。相邻边界参考固定全局维度、VPP、微批和重计算，其 3 次成功均在 MiMo-7B。

Nemotron-H-4B 中，完整系统将末端区间从 6 个模块减少为 1 个 Mamba2 和 1 个 MLP，其余模块迁入 V5–V7，第 4 次运行达到 58687 token/s。移除执行证据时，按数量生成的布局 [6, 8, 6, 8, 6, 6, 6, 6] 达到 50222 token/s，随后提高至 54480 token/s，仍未达标。

执行证据的上下文组织另在 Moonlight 上采用三轮训练对照，具体配置、容量失败及测量费用见附录 D。该项实验与表 5的多模型组件比较分别统计。

表 5　原型系统的智能体组件比较归档。结果质量按全部 9 次搜索统计；费用按达标搜索统计。

	全部搜索的结果		达标搜索的平均费用	
设置	吞吐提升/%	达标数量	训练次数	GPU 小时
完整系统	**11.7**	**9/9**	5.3	2.44
移除独立分析	6.2	4/9	8.0	3.52
合并智能体职责	4.3	2/9	9.5	4.18
移除执行证据	2.5	0/9	—	—
固定参数的启发式参考				
相邻边界移动	4.8	3/9	10.0	4.42

提升相对默认吞吐，按全部 9 次搜索统计；训练次数与 GPU 小时按达标搜索统计，各设置同时报告达标数量。统计来源见第 5.8节。

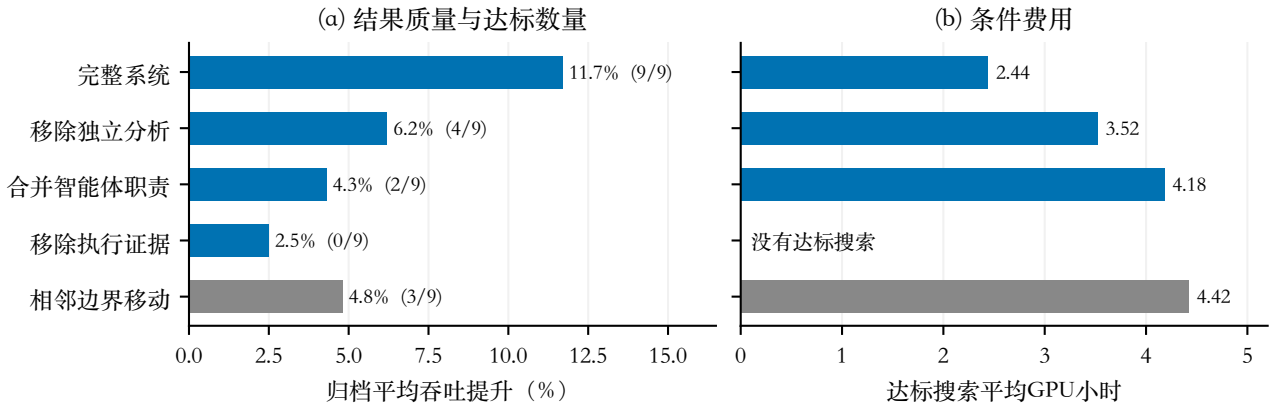


图 5　智能体组件的结果质量与条件费用。左图报告平均吞吐提升及达标数量，右图按达标搜索统计平均 GPU 小时。

5.5　执行信息与布局排序

阶段负载与完整时间的排序差异。

Qwen3-14B 的 43 项布局预测提供 854 对非并列比较，其中 257 对的最大阶段工作量排序与完整 DAG 排序相反。均匀布局与联合布局 [4, 6, 5, 5, 5, 5, 6, 4] 具有相同任务数量和 P2P 发送次数，局部工作量与完整时间仍有相反排序。附录表 D.1给出三项布局的通信暴露、预测时间和显存模型数值。

5.6　性能模型的留出验证

留出配置为 Qwen3-14B TP2/PP8/DP1/VPP1，排除该配置的 Profile 和整步锚点。预测使用同一成本库中的 TP2/PP4/DP2 组件记录。原选择采用 VPP2 的嵌套算子计时，按采集范围匹配后采用 VPP1 的整段计时，预测由 2055.57 变为 1539.20 ms；训练记录为 1510.26 ms，绝对百分比误差由 36.11% 降至 1.92%。该独立配置的另一份运行记录对应误差 1.74%。TP8/PP2 的组件预测误差为 19.33%，PP4 返回适用通信成本需求。

5.7　联合策略的训练案例

Qwen3-14B 的一条独立 BF16 训练轨迹展示了执行证据对后续提议的作用。智能体从已完成分区优化的父策略继续，查询相邻区间的反向任务，识别到区域 6 具有较高的局部执行代价。随后它将模块 30 迁至区域 5，同时调整 MLP 重计算，形成一项关联模块位置与存储设置的联合修改。

该策略的吞吐从父策略的 12608.80 提高至 13980.51 token/s，独立重复为 14043.38 token/s。提议中的模块、接收位置及重计算设置均保存在原始响应中。附录 B按执行顺序列出父策略、智能体修改、测量结果与后续处理。

图 6进一步区分各轮实测候选与当前最好策略。Qwen3-14B 的第二轮候选为 12592.99 token/s，结果保留 12608.80 token/s 的父策略，第三轮联合修改随后取得更高吞吐。MiMo-7B 在首轮取得 14264.12 token/s，第二轮候选降至 13657.22 token/s，已有最好结果继续保留。Qwen3-8B 的独立轨迹则将 TP8/PP2 共同调整为 TP4/PP4，吞吐从 10060.46 提高到 15127.87 token/s。各面板使用各自固定负载及初始策略。

Lightning 另有六轮完成的容量修复调查，包含并行度、FSDP 及优化器卸载的联合尝试。其执行结果用于修订后续提议，尚未产生有效吞吐；这条轨迹与图中的历史布局配对分别登记，完整策略及结果见附录表 B.3。

5.8　实验范围与复现材料

九模型与组件表格的归档口径见附录 E。四模型配对测量已核对 128 份逐 Rank 输出，Lightning 的独立布局比较另核对 72 份配置、计时、代码摘要及退出记录。预测搜索按成本来源、完成时间和容量状态保存候选。附录将完整配置、Agent 原始提议与工具执行结果放在同一轨迹中，独立重复使用相同策略身份登记。

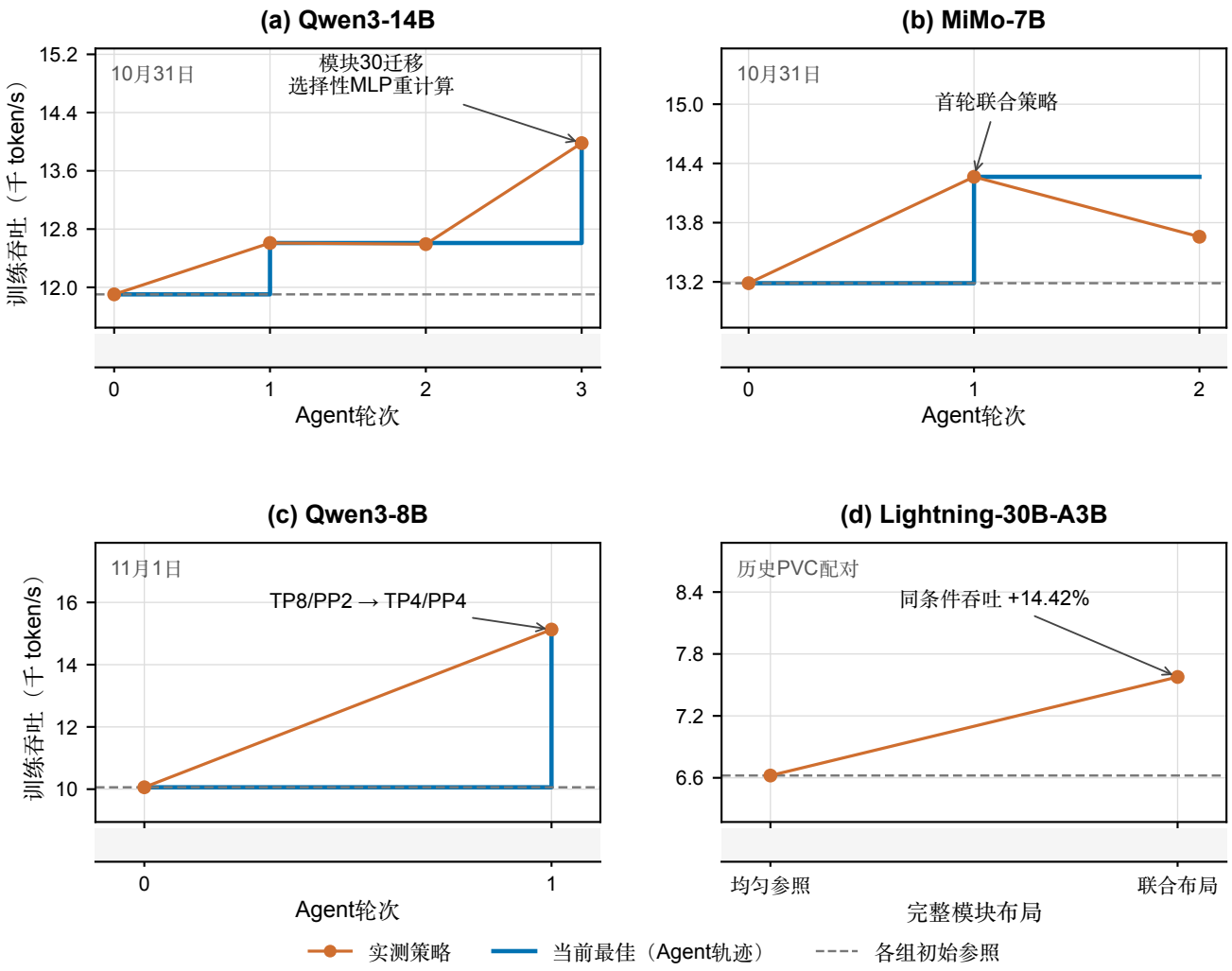


图 6 真实策略试验与结果保留。前三项为 BF16、16 卡、序列 2048、全局批 16，预热 5 步、测量 30 步的 Agent 轨迹，横轴为实际完成轮次；橙色表示实测候选，蓝色表示已有最好策略。右下为 Lightning 独立布局配对，序列 4096，预热 10 步、测量 30 步，横轴为布局类别。各点独立运行一次。

6 结论

ParaMeld 将训练依赖分析与多智能体联合规划结合。资源感知训练依赖图关联任务、通信参与者和存储对象，瓶颈关联子图与反事实回放为策略选择提供执行依据。智能体围绕这些对象联合修改并行参数、模块分配和运行设置，通过原生约束生成完整候选，并根据执行差异、成本敏感性与训练结果继续调整。四模型配对测量记录了联合策略的性能收益，逐轮轨迹进一步呈现了模块迁移、重计算修订及当前最好策略的保留过程。

参考文献

[1] NARAYANAN D, SHOEYBI M, CASPER J, et al. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM [C/OL]//Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2021: 58:1-58:15. https://doi.org/10.1145/3458817.3476209.

[2] RAJBHANDARI S, LI C L, YAO Z W, et al. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale[C/OL]//Proceedings of Machine Learning Research: v.162. 2022: 18332-18346. https://proceedings.mlr.press/v162/rajbhandari22a.html.

[3] JIA Z H, ZAHARIA M, AIKEN A. Beyond Data and Model Parallelism for Deep Neural Networks[C/OL]//Proceedings of Machine Learning and Systems: v.1. 2019: 1-13. https://proceedings.mlsys.org/paper_files/paper/2019/hash/b422680f3db0986ddd7f8f126baaf0fa-Abstract.html.

[4] TARNAWSKI J, NARAYANAN D, PHANISHAYEE A. Piper: Multidimensional Planner for DNN Parallelization[C/OL]//Advances in Neural Information Processing Systems: v.34. 2021: 24829-24840. https://proceedings.neurips.cc/paper/2021/hash/d01eeca8b24321cd2fe89dd85b9beb51-Abstract.html.

[5] ZHENG L M, LI Z H, ZHANG H, et al. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning[C/OL]//16th USENIX Symposium on Operating Systems Design and Implementation. 2022: 559-578. https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin.

[6] MIAO X P, WANG Y J, JIANG Y H, et al. Galvatron: Efficient Transformer Training over Multiple GPUs Using Automatic Parallelism [J/OL]. Proceedings of the VLDB Endowment, 2022, 16(3): 470-479. https://doi.org/10.14778/3570690.3570697.

[7] LIU G D, MIAO Y S, LIN Z Q, et al. Aceso: Efficient Parallel DNN Training through Iterative Bottleneck Alleviation[C/OL]//Proceedings of the Nineteenth European Conference on Computer Systems. 2024: 163-181. https://doi.org/10.1145/3627703.3629554.

[8] ZHU Z D, GIANNOULA C, ANDOORVEEDU M, et al. Mist: Efficient Distributed Training of Large Language Models via Memory-Parallelism Co-Optimization[C/OL]//Proceedings of the Twentieth European Conference on Computer Systems. 2025: 1298-1316. https://doi.org/10.1145/3689031.3717461.

[9] CHEN S, TANG Z, ZHANG W X, et al. OptiCo: Adaptive Distributed Training Optimization via Collaborative Agent Reasoning [C/OL]//Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2026: 27847-27867. https://doi.org/10.18653/v1/2026.acl-long.1283.

[10] WEI A J, NIE A, TEIXEIRA T S F X, et al. Improving Parallel Program Performance with LLM Optimizers via Agent-System Interfaces [C/OL]//Proceedings of Machine Learning Research: v.267. 2025: 66155-66177. https://proceedings.mlr.press/v267/wei25j.html.

[11] RAJBHANDARI S, RASLEY J, RUWASE O, et al. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models[C/OL]//SC20: International Conference for High Performance Computing, Networking, Storage and Analysis. 2020. https://doi.org/10.1109/SC41405.2020.00024.

[12] SUN Z B, CAO H Q, WANG Y W, et al. AdaPipe: Optimizing Pipeline Parallelism with Adaptive Recomputation and Partitioning [C/OL]//Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3. 2024: 86-100. https://doi.org/10.1145/3620666.3651359.

[13] HUANG Y P, CHENG Y L, BAPNA A, et al. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism[C/OL]//Advances in Neural Information Processing Systems: v.32. 2019: 103-112. https://proceedings.neurips.cc/paper_files/paper/2019/hash/093f65e080a295f8076b1c5722a46aa2-Abstract.html.

[14] NARAYANAN D, HARLAP A, PHANISHAYEE A, et al. PipeDream: Generalized Pipeline Parallelism for DNN Training[C/OL]//Proceedings of the 27th ACM Symposium on Operating Systems Principles. 2019: 1-15. https://doi.org/10.1145/3341301.3359646.

[15] FAN S Q, RONG Y, MENG C, et al. DAPPLE: A Pipelined Data Parallel Approach for Training Large Models[C/OL]//Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. 2021: 431-445. https://doi.org/10.1145/3437801.3441593.

[16] NARAYANAN D, PHANISHAYEE A, SHI K Y, et al. Memory-Efficient Pipeline-Parallel DNN Training[C/OL]//Proceedings of Machine Learning Research: v.139. 2021: 7937-7947. https://proceedings.mlr.press/v139/narayanan21a.html.

[17] QI P H, WAN X Y, HUANG G X, et al. Zero Bubble (Almost) Pipeline Parallelism[C/OL]//International Conference on Learning Representations. 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/d5a8e37f38a08c68162452dcba89ae9c-Abstract-Conference.html.

[18] LI D C, WANG H Y, XING E P, et al. AMP: Automatically Finding Model Parallel Strategies with Heterogeneity Awareness[C/OL]//Advances in Neural Information Processing Systems: v.35. 2022. https://doi.org/10.52202/068431-0480.

[19] UM T, OH B, KANG M, et al. Metis: Fast Automatic Distributed Training on Heterogeneous GPUs[C/OL]//2024 USENIX Annual Technical Conference. 2024: 563-578. https://www.usenix.org/conference/atc24/presentation/um.

[20] STRATI F, ZHANG Z D, MANOS G, et al. Sailor: Automating Distributed Training over Dynamic, Heterogeneous, and Geo-distributed Clusters[C/OL]//Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles. 2025: 204-220. https://doi.org/10.1145/3731569.3764839.

[21] LIN Z Q, MIAO Y S, ZHANG Q L, et al. nnScaler: Constraint-Guided Parallelization Plan Generation for Deep Learning Training [C/OL]//18th USENIX Symposium on Operating Systems Design and Implementation. 2024: 347-363. https://www.usenix.org/conference/osdi24/presentation/lin-zhiqi.

[22] WANG I, VENKATA V V, KRISHNAMURTHY A, et al. NEST: Network- and Memory-Aware Device Placement for Distributed Deep Learning[C/OL]//Proceedings of Machine Learning and Systems: v.8. 2026. https://proceedings.mlsys.org/paper_files/paper/2026/hash/949c07bf8756d51642c40b541c7306a8-Abstract-Conference.html.

[23] HU W F, CHEN L S, YUAN M, et al. Tessera: A Holistic Pipeline Parallelism Framework for Trillion-Parameter Heterogeneous MoE Training (Operational Systems)[C/OL]//20th USENIX Symposium on Operating Systems Design and Implementation. 2026: 287-304. https://www.usenix.org/conference/osdi26/presentation/hu-weifang.

[24] REN J, RAJBHANDARI S, AMINABADI R Y, et al. ZeRO-Offload: Democratizing Billion-Scale Model Training[C/OL]//2021 USENIX Annual Technical Conference. 2021: 551-564. https://www.usenix.org/conference/atc21/presentation/ren-jie.

[25] YANG H M, ZHOU J, FU Y, et al. ProTrain: Efficient LLM Training via Automatic Memory Management[C/OL]//Proceedings of Machine Learning and Systems: v.8. 2026. https://proceedings.mlsys.org/paper_files/paper/2026/hash/56280ad5fb53967fd55e4ba1b1cfc418-Abstract-Conference.html.

[26] GALE T, NARAYANAN D, YOUNG C, et al. MegaBlocks: Efficient Sparse Training with Mixture-of-Experts[C/OL]//Proceedings of Machine Learning and Systems: v.5. 2023. https://proceedings.mlsys.org/paper_files/paper/2023/hash/5a54f79333768effe7e8927bcccffe40-Abstract-mlsys2023.html.

[27] DUAN J F, LI X H, XU P, et al. Proteus: Simulating the Performance of Distributed DNN Training[J/OL]. IEEE Transactions on Parallel and Distributed Systems, 2024, 35(10): 1867-1878. https://doi.org/10.1109/TPDS.2024.3443255.

[28] RASHIDI S, SRIDHARAN S, SRINIVASAN S, et al. ASTRA-SIM: Enabling SW/HW Co-Design Exploration for Distributed DL Training Platforms[C/OL]//2020 IEEE International Symposium on Performance Analysis of Systems and Software. 2020: 81-92. https://doi.org/10.1109/ISPASS48437.2020.00018.

[29] WANG X Z, LI Q X, XU Y C, et al. SimAI: Unifying Architecture Design and Performance Tuning for Large-Scale Large Language Model Training with Scalability and Precision[C/OL]//22nd USENIX Symposium on Networked Systems Design and Implementation. 2025: 541-558. https://www.usenix.org/conference/nsdi25/presentation/wang-xizheng-simai.

[30] BANG J, CHOI Y, KIM M, et al. vTrain: A Simulation Framework for Evaluating Cost-Effective and Compute-Optimal Large Language Model Training[C/OL]//2024 57th IEEE/ACM International Symposium on Microarchitecture. 2024: 153-167. https://doi.org/10.1109/MICRO61859.2024.00021.

[31] LIANG M Y, KASSA H T, FU W Y, et al. Lumos: Efficient Performance Modeling and Estimation for Large-scale LLM Training [C/OL]//Proceedings of Machine Learning and Systems: v.7. 2025. https://proceedings.mlsys.org/paper_files/paper/2025/hash/a66caa1703fe34705a4368c3014c1966-Abstract-Conference.html.

[32] ISAEV M, MCDONALD N, DENNISON L, et al. Calculon: a Methodology and Tool for High-Level Codesign of Systems and Large Language Models[C/OL]//Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2023: 71:1-71:14. https://doi.org/10.1145/3581784.3607102.

[33] ZHANG H, FENG Y Z, CHEN Z W, et al. Communication Frequency in Megatron-LM: Experimental Insights Applied to Heterogeneous Distributed Training Time Prediction[J/OL]. Concurrency and Computation: Practice and Experience, 2026, 38(2): e70500. https://doi.org/10.1002/cpe.70500.

[34] YAO S Y, ZHAO J, YU D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C/OL]//International Conference on Learning Representations. 2023. https://openreview.net/forum?id=WE_vluYUL-X.

[35] SHINN N, CASSANO F, GOPINATH A, et al. Reflexion: Language Agents with Verbal Reinforcement Learning[C/OL]//Advances in Neural Information Processing Systems: v.36. 2023: 8634-8652. https://doi.org/10.52202/075280-0377.

[36] LIU S Y, GAO C, LI Y. AgentHPO: Large Language Model Agent for Hyper-Parameter Optimization[C/OL]//Proceedings of Machine Learning Research: v.280. 2025: 1146-1169. https://proceedings.mlr.press/v280/liu25c.html.

[37] LIU T, ASTORGA N, SEEDAT N, et al. Large Language Models to Enhance Bayesian Optimization[C/OL]//International Conference on Learning Representations. 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/84b8d9fcb4e262fcd429544697e1e720-Abstract-Conference.html.

[38] AKIBA T, SANO S, YANASE T, et al. Optuna: A Next-generation Hyperparameter Optimization Framework[C/OL]//Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2019: 2623-2631. https://doi.org/10.1145/3292500.3330701.

[39] EGERSDOERFER C, CARNS P, SNYDER S, et al. STELLAR: Storage Tuning Engine Leveraging LLM Autonomous Reasoning for High Performance Parallel File Systems[C/OL]//Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2025: 1252-1266. https://doi.org/10.1145/3712285.3759887.

[40] YE H G, XU H, YAN A, et al. Large Language Model-driven Large Neighborhood Search for Large-Scale MILP Problems[C/OL]//Proceedings of Machine Learning Research: v.267. 2025: 72131-72180. https://proceedings.mlr.press/v267/ye25j.html.

表 A.1　四模型配对测量的完整关键配置。并行组合依次为 TP/PP/CP/DP/EP/ETP/VPP，MBS 为微批大小。

模型	策略	并行组合	MBS	模块布局	重计算
MiMo-7B	初始	2/2/1/4/1/2/1	1	[18, 18]	完全
	ParaMeld	2/2/1/4/1/2/2	1	[9, 9, 9, 9]	注意力核心
Qwen3-8B	初始	8/2/1/1/1/1/1	1	[18, 18]	完全
	ParaMeld	4/2/1/2/1/4/1	1	[18, 18]	注意力核心、MLP
Qwen3-14B	初始	1/8/1/2/1/1/1	1	[5, 5, 5, 5, 5, 5, 5, 5]	完全
	ParaMeld	4/4/1/1/1/4/1	1	[10, 10, 10, 10]	注意力核心、MLP
Moonlight	初始	1/8/1/2/1/1/1	1	[4, 4, 4, 3, 3, 3, 3, 3]	完全
	ParaMeld	1/8/1/2/1/1/1	2	[3, 4, 4, 4, 3, 3, 3, 3]	完全

A　训练配置与测量协议

四模型 FP16 实验固定 16 张 GPU、序列 2048、全局批量 16、模型检查点及数据。计算与模型参数使用 FP16，主参数副本、主梯度和 Adam 一阶、二阶状态使用 FP32。每项配置独立运行一次，预热 5 次更新，测量后续 30 次更新；逐 Rank 记录检查全部 35 次优化器更新的成功状态及损失和梯度范数的有限性。吞吐由全局每次更新的 Token 数量除以最慢 Rank 的平均更新时间得到，显存取全部 Rank 的最大已分配值。

Agent 使用 `gpt-5.6-sol/medium`，从本次随机合法配置及其运行结果开始读取历史。完整执行图通过任务索引查询，直接上下文包含父策略、当前最好候选、最近结果及执行摘要。各项记录保留原始调用与实际生效配置；训练成功状态、吞吐测量与数值检查分别登记。

这些配置使用原始物理 Rank 映射、Megatron DDP 和分布式优化器，关闭 P2P 重叠，优化器 CPU 卸载比例为 0。MiMo 的 VPP2 配置使用微批分组 2。各模型的性能与显存见表 4。

CPU 预测搜索采用已有 PVC 组件成本，并独立记录完整预测、成本缺项及容量状态。一次提议可以产生多个完整候选；截止时选择已完成评价且符合该项接纳规则的最好结果。新的模型调用与同次调查的工具续接分别计数。在 120 秒搜索中，截止后的在途工作继续计入费用，其结果保留在轨迹中。

B　Agent 训练策略的实际修订

本例使用 Qwen3-14B、16 张 GPU、FineWeb、序列 2048、全局批 16、微批 1 和 BF16，固定 TP2/PP8/DP1/VPP1。每项配置预热 5 步、测量 30 步。表 B.1列出关键配置及一次相同策略复测，图 6保留三轮完成试验的候选顺序。

父策略完成测量后，Agent 查询区域 5 至 7 的反向执行。返回记录中，区域 6 的部分反向任务为 82.59 ms，相邻区域 5 和 7 分别为 68.66 与 70.90 ms，并包含跨微批依赖。Agent 提出将模块 30 由区域 6 迁至区域 5，同时修改 MLP

表 B.1　Qwen3-14B 训练轨迹。最后一行使用相同配置独立复测。

阶段	模块布局	重计算	吞吐 (token/s)
初始测量	[5, 5, 5, 5, 5, 5, 5, 5]	完全	11903.36
分区工具生成父策略	[3, 6, 6, 5, 5, 5, 6, 4]	完全	12608.80
Agent 联合修改	[3, 6, 6, 5, 5, 6, 5, 4]	选择性 MLP	13980.51
相同策略复测	[3, 6, 6, 5, 5, 6, 5, 4]	选择性 MLP	14043.38

表 B.2　Lightning 同代码、同负载的布局测量。

策略	完整模块布局	吞吐 (token/s)	显存 (GiB)
均匀参照	[7, 7, 7, 7, 6, 6, 6, 6]	6622.13	28.09
非均匀布局	[7, 7, 7, 7, 7, 7, 9, 1]	7577.06	25.17

表 B.3　Lightning 容量修复搜索的 Agent 提议与执行结果。执行条目包含重试，失败条目没有吞吐测量。

轮次	同次响应中的主要联合修改	本轮执行记录
1	TP2/PP4/DP2/EP2/ETP2 及不同模块布局与映射	3 条：2 次 OOM、1 次失败
2	PP8 的参数、梯度和优化器状态分片；优化器卸载比例 50% 或 100%；并行与微批联动	4 条：1 次 OOM、3 次失败
3	PP16/DP1 及 50% 卸载；TP2/PP8/DP1 等关联组合	2 条：2 次 OOM
4	TP1/PP4/DP4/EP4 与 TP2/PP4/DP2/EP2/ETP1	1 条：1 次 OOM
5	PP4/EP4 的 FSDP 分片，以及 TP2/PP4 和 TP1/PP8 的 DDP 候选	无新增训练结果
6	PP8 的 50% 卸载及无卸载候选，TP2/PP4 的 100% 卸载候选	无新增训练结果

重计算。核验工具保持其他模块的顺序与并行条件，生成完整配置并执行训练。该候选达到 13980.51 token/s，峰值已分配显存为 20.50 GiB；后续相同配置的独立复测为 14043.38 token/s。

同次提议还包含微批 2 与 TP4/PP4 调查。这两项缺少匹配成本，分别登记测量需求，未取得可用于候选排序的完整时间。控制实验随后在相同布局上关闭重计算，两次测量的平均吞吐为 17138.59 token/s、显存为 27.22 GiB；VPP2 无重计算配置发生 OOM。这些控制配置保留独立来源，与 Agent 原始提议分开统计。

B.1　Lightning 的同条件布局复核

Lightning-30B-A3B 的配对记录固定 16 张 GPU、TP2/CP1/PP4/DP2/EP2/VPP2、序列 4096、全局批 16 及微批 1，保留 BF16、完全重计算、优化器 CPU 卸载和一个 MTP 模块。两项配置的源码摘要一致，请求中的变化为模块区间；各项预热 10 步、测量 30 步，取得全部 16 个 Rank 的计时及两节点正常退出记录。

配对吞吐提高 14.42%，两项均为一次独立运行。72 份原始配置、逐 Rank 计时、代码摘要和退出文件已交叉核对。该配对支持固定并行组合下的布局收益；表 2保留原型归档口径，其 Lightning 三项汇总尚未与本次检查的原始目录建立对应。

B.2　Lightning 的多轮容量修复调查

该独立搜索采用 `gpt-5.6-sol/medium`，在本次加载及训练失败的基础上继续提出完整策略。六轮完成的响应各包含三项联合提议，共 18 项；第七轮停在分析阶段，未形成新策略。表 B.3按原始轮次列出主要修改及实际执行记录。所有提议均使用完全重计算，其他并行与存储选项在对应轮次中共同改变。

这些轮次保存了 Agent 根据失败证据提出的分片、卸载、模块分配和并行度修改，共有 10 条执行结果，其中 6 条 OOM、4 条执行失败。部分提议在约束与评价阶段结束，未进入新的训练测量。该搜索尚无有效吞吐记录，容量修复状态与历史已测布局的性能分别报告。

C　一次 Agent 提议的预测搜索展开

本例采用 Nemotron-H-4B 的 8 卡成本、TP2/CP1/PP4/DP1/VPP2、序列 2048、全局批 16、微批 1 和无重计算。52 个有序模块包括 24 个 Mamba2、24 个 MLP 与 4 个 Attention；初始区间为 [7, 7, 7, 7, 6, 6, 6, 6]，完整预测为 1671.00 ms。

表 C.1 同一次 Agent 响应的联合提议与工具执行。时间为完整预测值。

Agent 选择	依据与开放变量	申请评价/秒	工具执行与结果
PP4/DP1/VPP2，P2P 重叠开启	联合开放模块 21、27、46，减轻物理阶段 3 共同持有的两个区间	5 次/40 秒	直接布局 [7, 7, 8, 5, 7, 6, 7, 5]，1386.50 ms
PP2/DP2/VPP2，P2P 重叠关闭	减少流水线阶段，重新构造全部边界并计入 DP 同步	3 次/25 秒	生成 [13, 13, 13, 13]，返回适用成本缺项
PP4/DP1/VPP1，P2P 重叠关闭	调查取消虚拟交错及区间切换后的执行代价	3 次/25 秒	路径求解生成 [15, 12, 17, 8]，1194.99 ms

Agent 使用 `gpt-6.1-sol/low`。一次调用读取 Rank6 计算占用 1590.70 ms 及模块 21、27、46 的迁移代理，生成表 C.1中的三项调查。配额列保存 Agent 申请值，控制器按全局剩余时间核定实际执行配额。

Agent 直接生成第一项完整布局，并决定后两项的并行条件和开放范围。第三项的完整边界由路径求解器产生，在追加搜索开始后的 116.72 秒完成，成为该次截止前最好预测候选。相同重复编号的确定性路径搜索得到 1223.16 ms。候选的容量状态为需要显存测量，预测搜索未新增 GPU 训练。

三次独立 Agent 搜索分别得到 1217.96、1194.99 和 1386.50 ms；前两项均被各自运行保留，优于对应确定性搜索的 1223.16 ms。第三次也提出 VPP1 调查，其 [13, 13, 15, 11] 候选在 125.24 秒完成，预测 1320.96 ms，超出 120 秒截止时间。该次确定性搜索进入 VPP3 条件并得到 1223.16 ms。不同调查次序、运行设置和边界构造共同影响有限预算内的结果。

本例的三项调查来自一次真实模型响应。工具执行过程中的路径更新和后续边界求解单独登记，候选来源保持为智能体直接布局、原生参照或路径重构。后续模型调用读取执行反馈时，另建提议记录。

D 执行目标与上下文组织的检查

D.1 局部工作量与完整执行时间

表 D.1给出 Qwen3-14B 的三项布局。各布局均包含 1545 项任务及 448 项 P2P 发送。局部工作量为最大阶段逐模块前后向成本之和，显存为模型预测。

表中第二项的最大阶段工作量高于第一项，完整预测时间更短。路径构造使用区间在调度中的位置与前后向任务次数，完整回放继续检查通信就绪及资源次序变化。

D.2 Moonlight 上下文组织的训练轨迹

本项固定每组共同初始策略、源码、可用工具及三轮决策，比较结构化执行视图与直接记录两种上下文组织。表 D.2保留三条完成轨迹。GPU 活动费用包括训练、加载与剖析；资源保留费用还包括 16 卡已分配期间的模型调用、工具操作及结果收集。

执行视图轨迹完成两项加载验证，完整候选发生 CUDA OOM，结果保留初始已测配置。首条直接记录轨迹选择

表 D.1 Qwen3-14B 完整布局的预测排序。

完整布局	局部/ms	通信暴露/ms	完整时间/ms	峰值/GiB
[5, 5, 5, 5, 5, 5, 5, 5]	55.37	10.42	1534.47	24.60
[4, 6, 5, 5, 5, 5, 6, 4]	63.06	14.90	1488.96	21.80
[3, 7, 5, 5, 5, 5, 5, 5]	71.03	10.54	1533.69	24.40

表 D.2 Moonlight 三轮调查的候选测量与费用。吞吐单位为 token/s，费用单位为 GPU 小时。

上下文与重复	状态	初始吞吐	新候选吞吐	活动费用	保留费用
执行视图，1	完成	14427.89	OOM	0.787	4.011
直接记录，1	完成	14427.89	17082.02	1.089	3.922
直接记录，2	完成	14352.01	14907.81	1.545	4.560

TP1/PP8/DP2/VPP2、微批 1、完全重计算与 16 个虚拟分块。第二条选择 TP2/PP2/DP4、EP4/ETP1、VPP2、微批 1、完全重计算及 P2P 重叠。本组轨迹记录了两种上下文下的实际策略与费用，尚不足以确定上下文组织的稳定性能差异。

E 历史并行配置与原型汇总口径

表 E.1对应图 2。8 卡组固定序列 2048、全局批 16、微批 1、BF16 及完全重计算，使用相同训练入口，每项预热 5 次、测量 10 次，独立运行一次。12 卡组固定序列 1024、全局批 16、微批 1、TP1/PP6，采用相同入口与启动脚本的重复 2 至 5，每项预热 20 次、测量 100 次。

8 卡组的完整模块区间依次为 [5, 5, 5, 5, 4, 4, 4, 4]、[9, 9, 9, 9]、[18, 18]、[18, 18]、[36] 及 [36]。12 卡组的 CP 变化同时改变 DP，VPP 变化同时改变原生 P2P 设置。8 卡组未记录 GPU 型号及底层软件版本，比较条件由相同代码、检查点、数据与训练负载确定。

正文九模型、系统比较和组件表格沿用原型归档数值。归档协议记录双节点 16 卡、预热 20 步、测量 50 步及每项 3 次运行，组件比较以默认吞吐的 1.08 倍为达标目标；训练次数和 GPU 小时按达标搜索求均值。当前归档与全部逐次运行尚未建立完整对应，因此这些表格的汇总与本次已核对的配对测量分别使用。新增路径目标及 Agent 调查选择的作用由具体候选与搜索轨迹检查。

表 E.1 历史并行配置的完整 Rank 计时。12 卡吞吐为 4 次运行均值及样本标准差。

GPU	TP	CP	PP	VPP	DP	P2P 方式	吞吐 (token/s)	GiB
8	1	1	8	1	1	无重叠	12767.86	28.79
8	2	1	4	1	1	无重叠	12029.35	22.33
8	2	1	2	1	2	无重叠	12069.75	27.06
8	4	1	2	1	1	无重叠	9155.40	19.33
8	4	1	1	1	2	无流水线传输	8554.30	26.69
8	8	1	1	1	1	无流水线传输	5678.54	19.17
12	1	1	6	1	2	批量，无重叠	13394.00 ± 11.90	26.82
12	1	1	6	3	2	重叠	13994.39 ± 66.24	29.08
12	1	2	6	1	1	批量，无重叠	10748.91 ± 63.53	24.12
12	1	2	6	3	1	重叠	13018.44 ± 39.61	26.11

F 历史布局测量与执行示意

图 F.1保留两组历史布局测量。MiMo 采用 8 卡、TP2/PP4/DP1/VPP2、序列 2048 及完全重计算，预热 2 步、测量 3 步；Nemotron 采用 16 卡、TP2/PP4/DP2/VPP2、序列 4096 及无重计算，预热 1 步、测量 30 步。各组固定全局批 16 与微批 1，每项配置独立运行一次。

图 F.2采用另一组 MiMo 配对测量，使用 16 卡、TP2/PP4/DP2/VPP2、序列 2048、全局批 16、微批 1 和完全重计算，预热 5 步、测量 10 步。图中迭代时间为 2230 与 1925 ms，任务宽度用于表示执行次序。

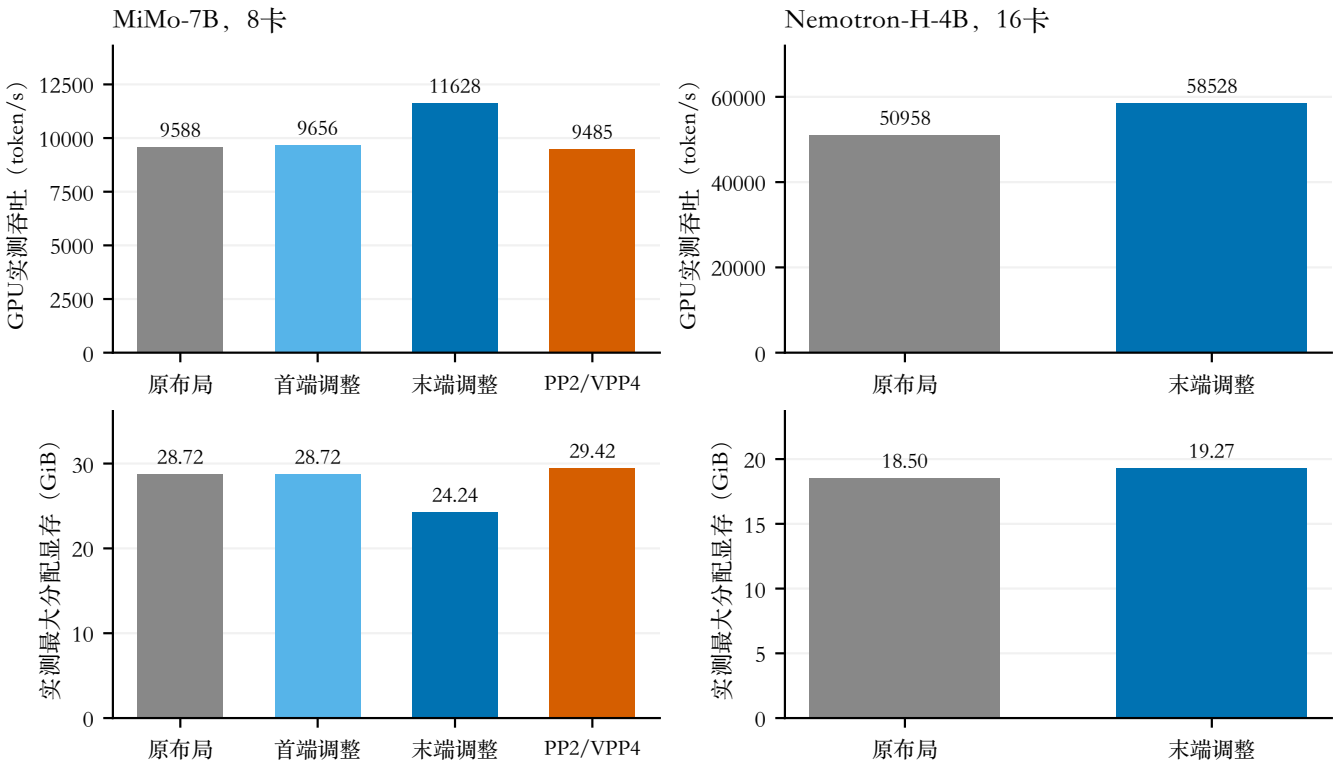


图 **F.1** 历史布局的吞吐与显存测量。两组使用各自的固定训练条件与测量窗口。

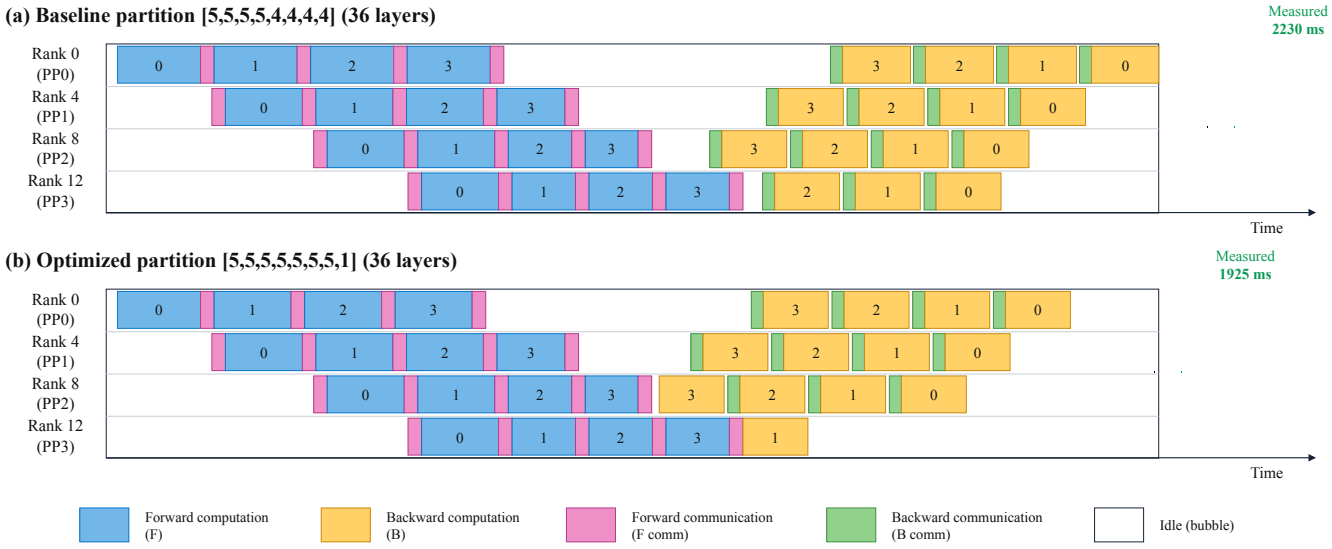


图 **F.2**　MiMo 布局修改前后的流水线执行示意。标注为该组配对运行的迭代时间。